RESPUESTAS DEL CUESTIONARIO TEÓRICO

1) Sistemas Operativos - Generalidades

a) ¿Quién es el encargado de administrar el tiempo de la CPU y cuál es su función principal dentro del sistema?

El **planificador de procesos** (también llamado **scheduler**) es el encargado de administrar el tiempo de la CPU.
Su función principal es **decidir qué proceso o hilo se ejecutará en la CPU en cada momento**, optimizando el uso de la misma según criterios como:

- **Equidad:** Todos los procesos deben tener acceso al procesador.
- **Eficiencia:** Mantener la CPU ocupada el mayor tiempo posible.
- **Tiempo de respuesta:** Minimizar el tiempo que tarda un proceso en empezar a ejecutarse.
- **Rendimiento:** Maximizar la cantidad de procesos completados por unidad de tiempo.

---

b) Explique a qué se denomina "Contexto de Ejecución". ¿Cuáles son las variables intervinientes?

El **Contexto de Ejecución** de un proceso o hilo es el **conjunto de información que el sistema operativo necesita guardar para poder reanudar la ejecución de ese proceso más adelante**, tal como estaba antes de ser interrumpido.

Las variables intervinientes incluyen:

1. **Registros de la CPU:**
    - Contador de programa (PC): indica la próxima instrucción a ejecutar.
    - Registros de propósito general (acumuladores, registros de índice, etc.)
    - Registro de estado (flags).
2. **Pila (Stack):**
    - Información relacionada con las llamadas a funciones, parámetros y variables locales.
3. **Estado del proceso:**
    - Ej: listo, ejecutándose, bloqueado.

4. **Información de gestión de memoria:**
    - Segmentos de código, datos y pila.
    - Tablas de páginas o segmentos.
5. **Recursos asignados:**
    - Archivos abiertos, dispositivos, etc.

Este contexto se **guarda en una estructura llamada PCB (Process Control Block)** cada vez que hay un cambio de contexto.

---

c) ¿A qué se denomina Sistema Operativo de Tiempo Real? ¿Por qué usar un Sistema Operativo de Tiempo Real?

Un **Sistema Operativo de Tiempo Real (RTOS)** es un sistema que está diseñado para ejecutar tareas **en tiempos predecibles y garantizados**, donde el cumplimiento de los plazos (deadlines) es crítico.

- **Existen dos tipos:**
    - **Duro:** Si no se cumple el tiempo límite, el fallo es crítico (ej: control de un marcapasos).
    - **Blando:** Es preferible cumplir el tiempo límite, pero no es catastrófico si no se logra (ej: transmisión de video).

¿Por qué usar un RTOS?

- Porque en ciertas aplicaciones **la precisión en los tiempos de respuesta es tan importante como el resultado mismo**.
- Se utilizan en:
    - Sistemas embebidos.
    - Automatización industrial.
    - Sistemas aeroespaciales.
    - Robótica.
    - Telecomunicaciones.

Un RTOS asegura que las tareas críticas se ejecuten dentro del tiempo requerido, garantizando **determinismo y predictibilidad**, cosa que un sistema operativo general (como Windows o Linux estándar) no puede garantizar.

2) Kernel de FreeRTOS

a) Explique y desarrolle las caracterısticas de una “Tarea” en FreeRTOS. Proponga un ejemplo de una tarea manteniendo la sintaxis correcta.

a) Características de una “Tarea” en FreeRTOS

Una Tarea en FreeRTOS es una función escrita por el usuario que se ejecuta concurrentemente con otras tareas. Cada tarea es gestionada por el kernel y tiene:

- Prioridad definida: determina su importancia frente a otras tareas.
- Pila propia: para almacenar contexto, variables locales, etc.
- Estado: activo, bloqueado, suspendido, etc.
- Parámetro de entrada: opcional para configuraciones.
- Planificación cooperativa o por prioridades.

```
#include "FreeRTOS.h"
#include "task.h"

// Definición de la tarea
void vTareaEjemplo(void *pvParameters) {
    for(;;) {
        // Código principal de la tarea
        printf("Tarea en ejecución\n");

        // Simula una espera de 500 ms
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

// Crear la tarea en main o en otra función
void app_main() {
    xTaskCreate(vTareaEjemplo, "Tarea1", 1000, NULL, 1, NULL);
    vTaskStartScheduler(); // Inicia el planificador
}
```

b) Explique y desarrolle los “Estados de una Tarea” y sus transiciones. Indique las funciones que permiten las transiciones.

b) Estados de una Tarea y Transiciones

En FreeRTOS, una tarea puede estar en uno de estos **estados:**

Estado	Descripción
Running	La tarea está ejecutándose en la CPU.
Ready	Lista para ejecutarse, esperando su turno.
Blocked	Esperando un evento o temporizador.
Suspended	Suspendida manualmente, no será ejecutada.
Deleted	Ha sido eliminada y no se ejecutará más.

Transiciones entre estados y funciones asociadas:

- **Ready → Running:** Automático por el scheduler si es la tarea de mayor prioridad.
- **Running → Blocked:** Por vTaskDelay(), xQueueReceive(), xSemaphoreTake().
- **Running → Ready:** Cuando una tarea desbloqueada tiene mayor prioridad.
- **Running → Suspended:** Por vTaskSuspend().
- **Suspended → Ready:** Por vTaskResume().
- **Running → Deleted:** Por vTaskDelete().

c) ¿A que nos referimos con el concepto de “tareas de ´ procesamiento continuo”? ¿y a que nos referimos ´ con el concepto de “tareas manejadas por eventos”?

c) Tareas de Procesamiento Continuo vs Tareas Manejadas por Eventos

- **Tareas de Procesamiento Continuo:**
  Ejecutan un bucle **sin bloqueos intencionales**, haciendo procesamiento constante o verificaciones rápidas.

Ejemplo:

```
for(;;) {
    LeerSensor();
```

```
}
```

- **Tareas Manejadas por Eventos:**
  Se bloquean esperando un evento como una señal, semáforo, o mensaje en cola, lo que optimiza el uso de CPU.

Ejemplo:

```
for(;;) {
    if (xSemaphoreTake(xSemaforo, portMAX_DELAY)) {
        ProcesarDato();
    }
}
```

d) ¿Cuales son los tipos de eventos por los que una ´ tarea puede estar esperando al entrar en el estado bloqueado? Explicar.

d) Tipos de Eventos por los que una Tarea puede estar en Estado Bloqueado

Una tarea entra en estado **bloqueado** cuando espera alguno de estos eventos:

1. **Temporizadores o retardos:**
   - vTaskDelay(), vTaskDelayUntil().
2. **Colas:**
   - xQueueReceive(): esperando datos en una cola.
3. **Semáforos o Mutex:**
   - xSemaphoreTake(), xSemaphoreGive().
4. **Notificaciones directas:**
   - ulTaskNotifyTake(), xTaskNotifyWait().

Cada evento tiene un tiempo de espera que puede ser **definido o infinito**.

e) ¿Cual es la funcion de la IDLE TASK? ¿Puede el procesador no estar ejecutando ninguna tarea en algun momento? Explicar. ´

e) Función de la IDLE TASK y ejecución sin tareas

- La **Idle Task** es una tarea que crea automáticamente el kernel.
- Su prioridad es la **más baja** y se ejecuta solo cuando **no hay ninguna tarea Ready**.

Funciones de la Idle Task:

- Liberar memoria de tareas eliminadas.
- Ejecutar el hook de bajo consumo (vApplicationIdleHook()).
- Mantener la CPU ocupada cuando no hay otras tareas.

¿Puede el procesador no estar ejecutando ninguna tarea?
No, porque si todas las tareas están bloqueadas o suspendidas, el sistema ejecutará la **Idle Task**. Si se configura, en este estado la CPU puede entrar en **modo de bajo consumo**.

3) Desarrolle como es el almacenamiento de datos en el recurso "Cola" proporcionado por el Kernel.

¿Qué es una *cola* en FreeRTOS?

Una cola es una estructura de datos proporcionada por el kernel para intercambiar datos entre tareas de forma segura y ordenada. Sigue el esquema FIFO (*First In, First Out*), es decir, el primer dato que entra es el primero en salir.

¿Cómo se almacena un dato en una cola?

1. Cuando se crea la cola con xQueueCreate(), el sistema reserva en memoria dinámica o estática:
    - Un buffer circular de tamaño total:
      número de elementos × tamaño de cada elemento
    - Una estructura de control para administrar la cola.
2. Cada dato se copia por valor, es decir, el contenido del dato se almacena en el buffer interno de la cola, no solo el puntero.

Ejemplo:

```
xQueueHandle xCola;
xCola = xQueueCreate(5, sizeof(int));
```

4) Explique utilizando un ejemplo con codigo el porque´es comun que el recurso Cola posea multiples tareas ´ escritoras y una sola tarea lectora. Explique el proceso de bloqueo por escritura y bloqueo por lectura en el recurso Cola.

4) ¿Por qué es común tener múltiples tareas escritoras y una sola tarea lectora en una cola?

Motivación:

- Múltiples tareas escritoras producen datos desde diferentes fuentes o eventos (sensores, interrupciones, entradas).
- Una sola tarea lectora se encarga de procesar todos esos datos de forma centralizada y ordenada.

Ventajas del patrón:

- Evita duplicación de lógica de procesamiento.
- Ordena el acceso a recursos comunes (pantalla, archivo, red).
- Reduce errores de concurrencia.

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include <stdio.h>

QueueHandle_t xCola;

// Tarea Escritora 1
void vTareaProductora1(void *pvParameters) {
    int dato = 100;
    for (;;) {
        xQueueSend(xCola, &dato, portMAX_DELAY); // Bloquea si cola llena
        printf("Tarea 1 escribió: %d\n", dato);
        vTaskDelay(pdMS_TO_TICKS(1000));
```

```
    }
}

// Tarea Escritora 2
void vTareaProductora2(void *pvParameters) {
    int dato = 200;
    for (;;) {
        xQueueSend(xCola, &dato, portMAX_DELAY); // Bloquea si cola llena
        printf("Tarea 2 escribió: %d\n", dato);
        vTaskDelay(pdMS_TO_TICKS(700));
    }
}

// Tarea Lectora (consumidor)
void vTareaConsumidora(void *pvParameters) {
    int recibido;
    for (;;) {
        if (xQueueReceive(xCola, &recibido, portMAX_DELAY)) { // Bloquea si
cola vacía
            printf("Tarea consumidora recibió: %d\n", recibido);
        }
    }
}

void app_main() {
    xCola = xQueueCreate(5, sizeof(int)); // Cola con espacio para 5
enteros

    xTaskCreate(vTareaProductora1, "Prod1", 1000, NULL, 2, NULL);
    xTaskCreate(vTareaProductora2, "Prod2", 1000, NULL, 2, NULL);
    xTaskCreate(vTareaConsumidora, "Consumidor", 1000, NULL, 1, NULL);

    vTaskStartScheduler();
}
```

Bloqueo por escritura y lectura

Bloqueo por escritura (cola llena):

- Si la cola está llena (ej. ya tiene 5 elementos) y una tarea llama a xQueueSend():
    - **La tarea escritora se bloquea** hasta que haya espacio disponible.
    - O, si se usa xTicksToWait = 0, **la escritura falla inmediatamente**.

Bloqueo por lectura (cola vacía):

- Si la cola está vacía y xQueueReceive() es llamado:
    - La tarea lectora **queda bloqueada** esperando hasta que llegue un dato.
    - O, si se usa xTicksToWait = 0, **la lectura falla inmediatamente**.

5) A la hora de enviar datos compuestos: explique como se lleva a cabo dicho proceso y por que no genera conflictos con la definicion de la macro del recurso. ¿Cual seria la implementacion si los datos fueran "grandes", lo que ocasionaria un problema con la memoria RAM?

5) Envío de Datos Compuestos en FreeRTOS

¿Qué son datos compuestos?

Son estructuras que agrupan varios tipos de datos en un solo bloque, como structs.

¿Cómo se envían datos compuestos por cola?

FreeRTOS permite enviar cualquier tipo de dato (int, float, struct, etc.) por cola siempre que se conozca su tamaño en el momento de crearla.

No hay conflicto con la macro xQueueCreate() porque esta macro permite definir:

```
xQueueCreate(n_items, size_item);
```

- n_items: cantidad de elementos a almacenar.
- size_item: tamaño de cada elemento en bytes → aquí colocamos sizeof(struct) si es un dato compuesto.

¿Y si los datos son "grandes"? (problemas de RAM)

Cuando los datos son muy pesados (ej. imágenes, buffers grandes, cadenas largas), no conviene copiarlos dentro de la cola por dos motivos:

1. La cola tendría que reservar un buffer grande por elemento.
2. Se pierde eficiencia por copiar datos grandes.

Solución: pasar punteros en lugar de estructuras completas

En vez de enviar el dato, se envía un puntero al dato, y se gestiona la memoria por fuera de la cola.

6) Explicar el funcionamiento del esquema de la fig 1:

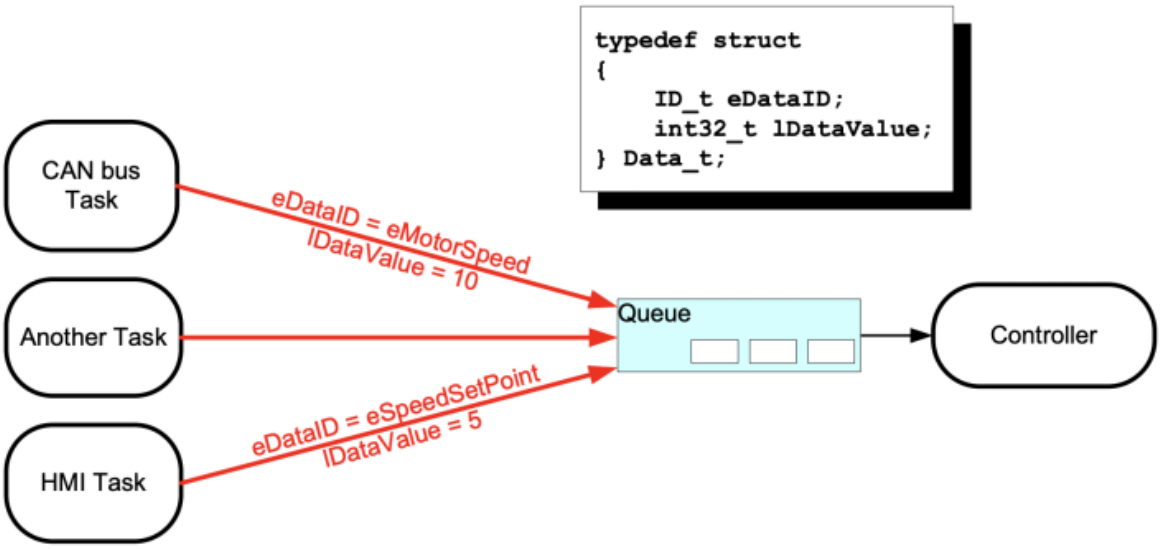


Fig. 1. Recibir datos de múltiples fuentes

6) Explicación del funcionamiento del esquema de la Fig. 1

El esquema muestra varias tareas productoras que envían información a una misma cola y una tarea Controller que actúa como consumidora. Todas las tareas utilizan el mismo tipo de dato, Data_t. El campo eDataID identifica qué representa el mensaje y lDataValue contiene su valor. Por ejemplo, la tarea CAN puede enviar la velocidad medida y la HMI puede enviar la referencia solicitada.

xQueueSend() copia cada estructura dentro del búfer de la cola. Por eso, la variable local de la tarea productora puede reutilizarse después del envío. La cola conserva el orden FIFO y bloquea de manera segura a las tareas cuando está llena. Controller queda bloqueada en xQueueReceive() mientras no haya datos; al recibir uno, utiliza eDataID para decidir cómo interpretar lDataValue.

```
typedef enum { MOTOR_SPEED, SPEED_SETPOINT } ID_t;

typedef struct {
    ID_t eDataID;
    int32_t lDataValue;
} Data_t;

void controller_task(void *parameters) {
    Data_t data;

    while (true) {
        xQueueReceive(data_queue, &data, portMAX_DELAY);

        if (data.eDataID == MOTOR_SPEED) {
            motor_speed = data.lDataValue;
        } else if (data.eDataID == SPEED_SETPOINT) {
            speed_setpoint = data.lDataValue;
        }
    }
}
```

Este patrón permite reunir datos de diferentes fuentes sin que las tareas productoras accedan directamente al controlador ni compartan variables sin protección.

7) Programa de ejemplo para comprender el uso de un semáforo mutex

Un mutex permite que una sola tarea por vez utilice un recurso compartido. En este ejemplo, dos tareas escriben por la misma UART. Cada tarea toma el mutex antes de transmitir y lo libera al terminar; de esta forma, los mensajes no se mezclan.

```
SemaphoreHandle_t uart_mutex;

void sensor_task(void *parameters) {
    while (true) {
        xSemaphoreTake(uart_mutex, portMAX_DELAY);
        uart_puts(UART_ID, "Medicion disponible\r\n");
        xSemaphoreGive(uart_mutex);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void alarm_task(void *parameters) {
    while (true) {
        xSemaphoreTake(uart_mutex, portMAX_DELAY);
        uart_puts(UART_ID, "Estado de proteccion\r\n");
        xSemaphoreGive(uart_mutex);
        vTaskDelay(pdMS_TO_TICKS(250));
    }
}
```

```
int main(void) {
    uart_mutex = xSemaphoreCreateMutex();
    xTaskCreate(sensor_task, "Sensor", 256, NULL, 2, NULL);
    xTaskCreate(alarm_task, "Alarma", 256, NULL, 2, NULL);
    vTaskStartScheduler();
}
```

El mutex debe mantenerse solamente durante el acceso a la UART. No conviene realizar retardos dentro de la sección crítica, porque las demás tareas permanecerían bloqueadas durante ese tiempo.

8) Programa de ejemplo para comprender la inversión de prioridad

La inversión de prioridad ocurre cuando una tarea de alta prioridad queda esperando un recurso retenido por una tarea de baja prioridad. Si una tarea de prioridad media interrumpe a la de baja, también retrasa indirectamente a la de alta.

El siguiente ejemplo usa una cola de un elemento como token de acceso a la UART. La tarea baja retira el token y comienza a transmitir. Luego la tarea alta intenta retirar el mismo token y queda bloqueada. Mientras tanto, la tarea media puede ejecutar repetidamente y evitar que la tarea baja termine y devuelva el token. Como una cola común no implementa herencia de prioridad, aparece la inversión.

```
QueueHandle_t uart_token;

void low_task(void *parameters) {
    uint8_t token;
    while (true) {
        xQueueReceive(uart_token, &token, portMAX_DELAY);
        uart_puts(UART_ID, "Mensaje largo de tarea baja\r\n");
        for (volatile uint32_t i = 0; i < 300000; i++) { }
        xQueueSend(uart_token, &token, portMAX_DELAY);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

void medium_task(void *parameters) {
    while (true) {
        process_data();
        taskYIELD();
    }
}

void high_task(void *parameters) {
    uint8_t token;
    while (true) {
        xQueueReceive(uart_token, &token, portMAX_DELAY);
        uart_puts(UART_ID, "Alarma prioritaria\r\n");
        xQueueSend(uart_token, &token, portMAX_DELAY);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

int main(void) {
    uint8_t token = 1;
    uart_token = xQueueCreate(1, sizeof(token));
    xQueueSend(uart_token, &token, 0);
    xTaskCreate(low_task, "Baja", 256, NULL, 1, NULL);
    xTaskCreate(medium_task, "Media", 256, NULL, 2, NULL);
    xTaskCreate(high_task, "Alta", 256, NULL, 3, NULL);
    vTaskStartScheduler();
}
```

La cola se usa aquí de forma deliberada para mostrar el problema. Si el recurso necesita exclusión mutua entre tareas, debe utilizarse xSemaphoreCreateMutex(), porque el mutex de FreeRTOS incorpora herencia de prioridad.

9) Herencia de prioridad

La herencia de prioridad es el mecanismo por el cual una tarea de baja prioridad que posee un mutex hereda temporalmente la prioridad de una tarea superior que está esperando ese mutex. Así puede finalizar su sección crítica sin ser desplazada por tareas intermedias. Cuando libera el mutex, recupera su prioridad original.

Sin herencia de prioridad

Tarea	t0	t1	t2	t3	t4	t5
Alta (P3)	Lista	Bloqueada	Bloqueada	Bloqueada	Ejecuta	Ejecuta
Media (P2)	Lista	Lista	Ejecuta	Ejecuta	Lista	Lista
Baja (P1)	Posee mutex	Lista	Lista	Libera mutex	Lista	Lista

Con herencia de prioridad

Tarea	t0	t1	t2	t3	t4	t5
Alta (P3)	Lista	Bloqueada	Bloqueada	Ejecuta	Ejecuta	Lista
Media (P2)	Lista	Lista	Lista	Lista	Lista	Ejecuta
Baja (P1→P3)	Posee mutex	Hereda prioridad	Ejecuta	Libera mutex	Lista	Lista

En FreeRTOS este comportamiento se obtiene con un mutex creado mediante xSemaphoreCreateMutex(). Un semáforo binario o una cola utilizada como token no ofrecen herencia de prioridad.

10) Programa de ejemplo para comprender el punto muerto

Un punto muerto o deadlock aparece cuando dos tareas esperan indefinidamente recursos que están retenidos entre sí. En el ejemplo, uart_token es una cola de un elemento que administra el uso de la UART e i2c_mutex protege el bus I²C.

```
QueueHandle_t uart_token;
SemaphoreHandle_t i2c_mutex;

void task_a(void *parameters) {
    uint8_t token;
    while (true) {
        xQueueReceive(uart_token, &token, portMAX_DELAY);
        vTaskDelay(pdMS_TO_TICKS(10));
        xSemaphoreTake(i2c_mutex, portMAX_DELAY);

        read_i2c_sensor();
        uart_puts(UART_ID, "Tarea A\r\n");

        xSemaphoreGive(i2c_mutex);
        xQueueSend(uart_token, &token, portMAX_DELAY);
    }
}

void task_b(void *parameters) {
    uint8_t token;
    while (true) {
        xSemaphoreTake(i2c_mutex, portMAX_DELAY);
        vTaskDelay(pdMS_TO_TICKS(10));
```

```
            xQueueReceive(uart_token, &token, portMAX_DELAY);

            read_i2c_sensor();
            uart_puts(UART_ID, "Tarea B\r\n");

            xQueueSend(uart_token, &token, portMAX_DELAY);
            xSemaphoreGive(i2c_mutex);
        }
    }
```

Si task_a obtiene primero uart_token y task_b obtiene primero i2c_mutex, task_a espera el mutex mientras task_b espera el token. Ninguna puede avanzar ni liberar lo que ya posee.

Para evitar el punto muerto se deben aplicar estas reglas:

- Tomar siempre los recursos en el mismo orden en todas las tareas.
- Usar tiempos de espera finitos y liberar el recurso ya adquirido si falla el segundo acceso.
- Mantener las secciones críticas cortas.
- Cuando sea posible, asignar cada periférico a una única tarea y comunicarse con ella mediante una cola.

La corrección más simple para el ejemplo es que ambas tareas tomen primero i2c_mutex y después uart_token. Al eliminar el orden circular de espera, el punto muerto no puede producirse.